# LLMs for vulnerability research

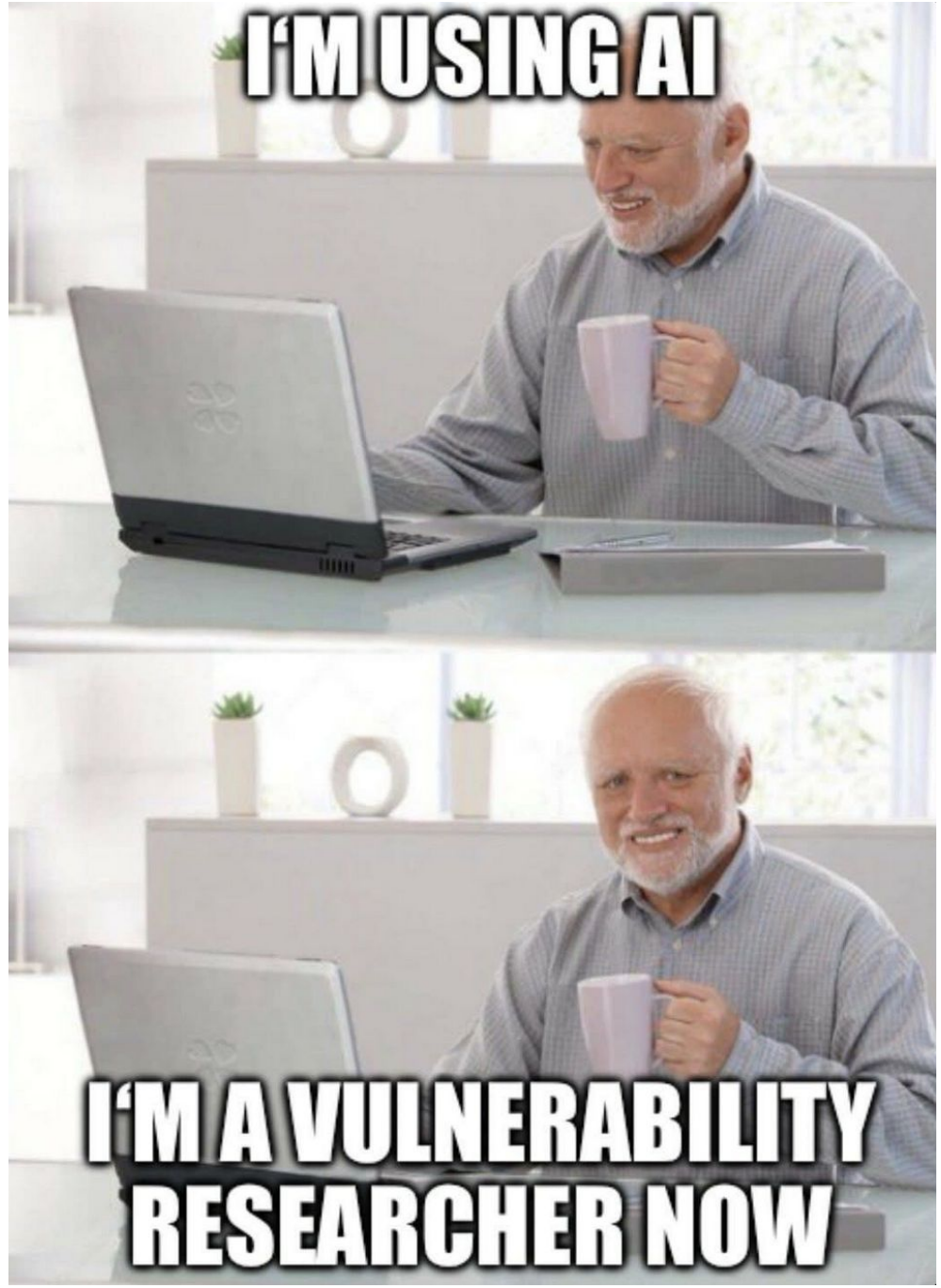
## building (and operating) a personal AI bug-hunting harness

Andrej Tomči

# Motivation, goals, results

# Funny little side-project

- I want to stay in touch with the latest AI developments
- BB and vulnresearch always seemed cool to me, but I never really took serious action
- Would be nice to combine, right?
- (and what is the magic behind XBOW, BugBunny.ai, etc?)


I'M USING AI
I'M A VULNERABILITY RESEARCHER NOW

# How I want it to work

- A-Z on it’s own, with minimum/no human interventions
  - Wake up -> launch review -> do work -> lunch break -> do work -> have reportable findings
- No false positives > some false negatives
- No opsec
  - Separate VM, trusted targets, whitebox, no interaction with live targets
- Quality outputs
  - Tool finds, tool reports, tool validates -> I copy&paste report -> profit?

# Results so far

- CVEs and valid vulns in:
  - Vim <3
  - Apache ActiveMQ
  - NetworkManager (RHEL)
  - Keycloak
  - Rancher, Argo CI/CD
  - Spotify Backstage
  - Sentry
  - Slovensko.Digital
  - Bruno, Gitea, WikiJS
  - Pi-hole, HomeAssistant
  - And many more (*upcoming*)!
- Also, *tons* of duplicates
  - Vercel, Angular (+ other Google projects), MongoDB, Grafana, …
  - more people had similar motivation ☺

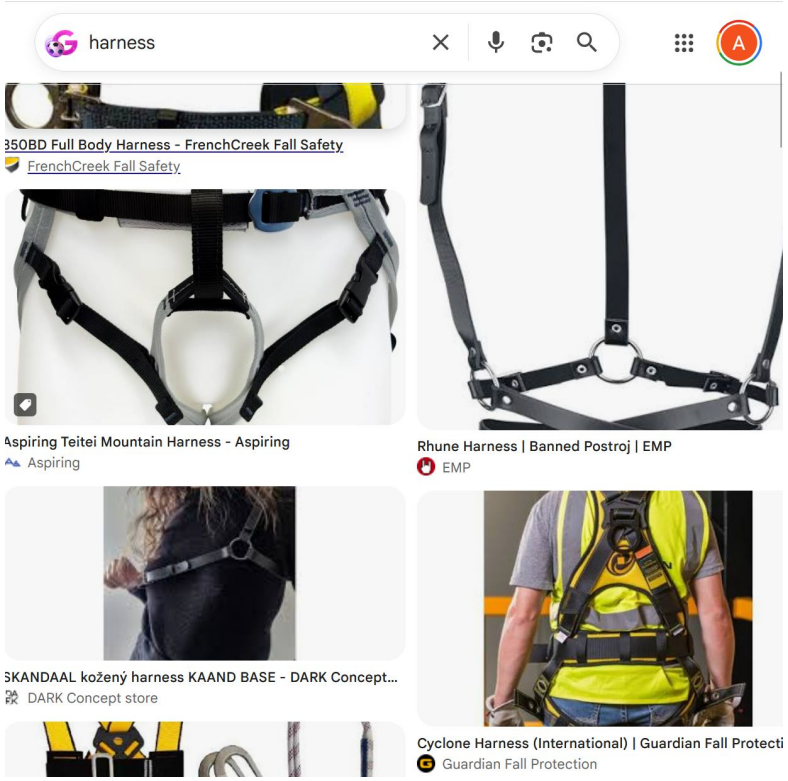
# Architecture & features

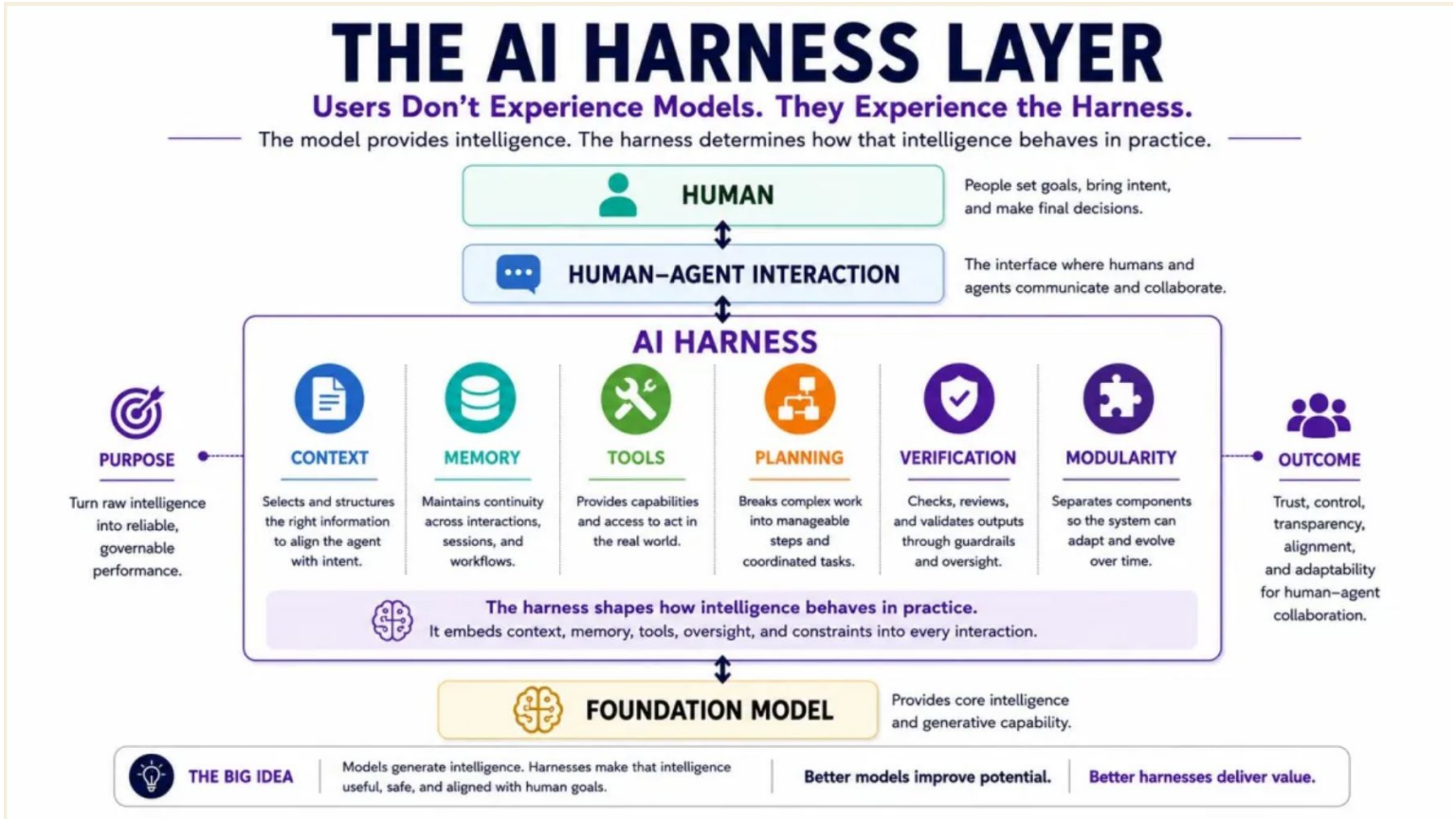

Always has been
It's just markdown files?

# So what is it, really?

- AI harness built on top of Claude Code
  - Commands, Skills
- + bunch of resources
  - Methodologies (WSTG, Portswigger, …)
  - Vulnerability writeups
- + bunch of (deterministic) tools
  - For vuln discovery (SAST, fuzz, …)
  - For making the model behave (scripts enforcing completeness, …)

# What is a harness/scaffolding?

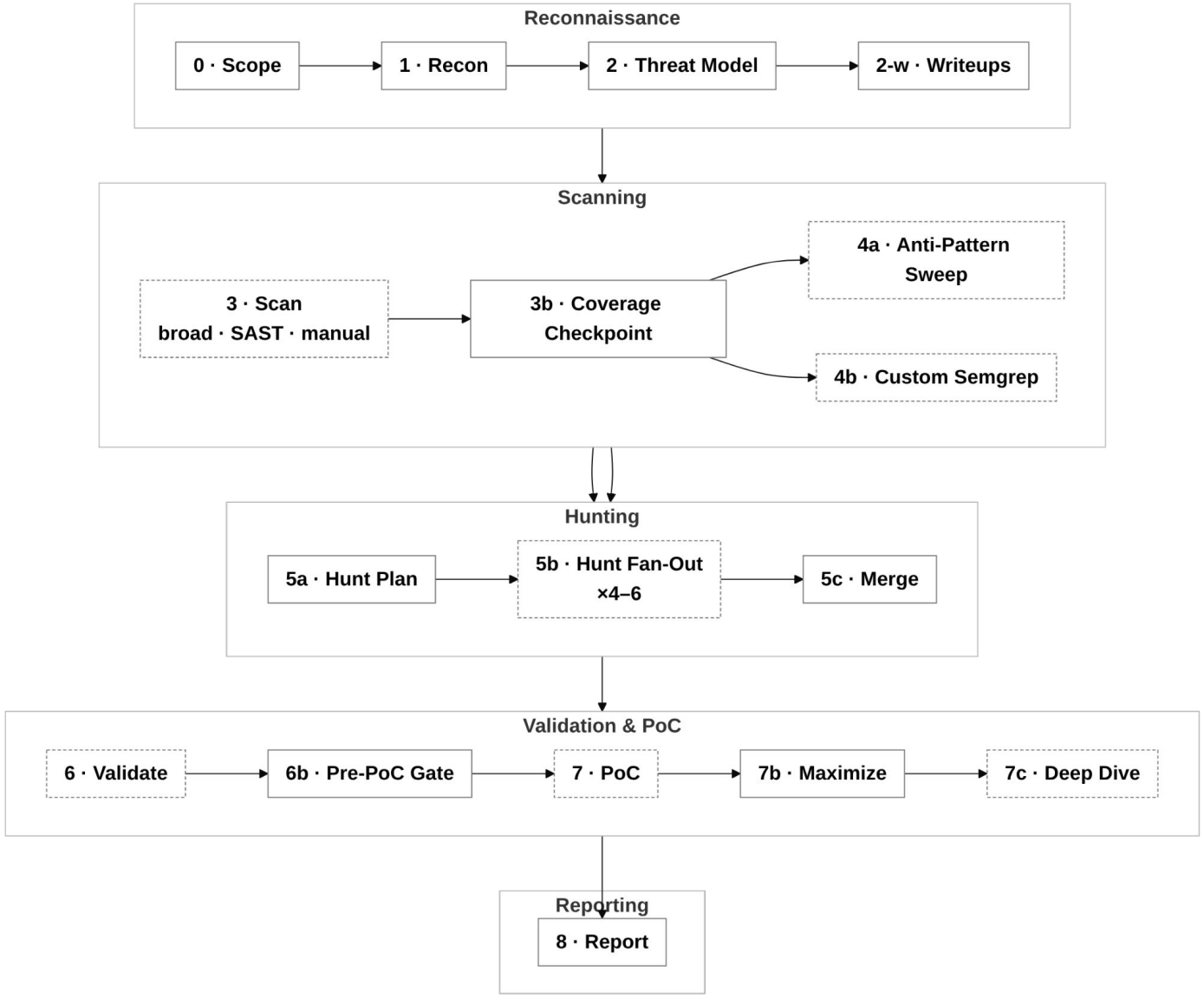


# Workflow

1. Open target repository in Claude code
2. Run /review
3. (wait for 2hrs)
4. Review results, ask follow-up questions, demand better coverage
5. Let it generate report & run QA
6. Submit report
7. … wait 6 months for a triage …

# What it does during the “(wait for 2hrs)”?



# Tool structure

```
bughunter/
├── commands/     ▸ the pipeline — 22 phase prompts
│   ├── review.md  ◂ orchestrator (runs all phases)
│   └── recon → threat_model → scan / broad_scan → checkpoint → antipattern / custom_semgrep
│        → hunt → validate → poc → maximize → deep_dive → report
├── agents/       ▸ fan-out workers — slice-hunter · hunt-merger · finding-validator
├── skills/       ▸ 18 knowledge packs
│   ├── method — semgrep-taint · finding-analysis · variant-hunting · advisory-writing …
│   └── corpus — vuln-class-cards · portswigger-academy · payload-crafting · hackdex …
├── scripts/      ▸ 32 deterministic helpers (no-LLM glue)
│   └── haiku-broad-scan · citation-drop · hybrid-dedup · reconcile-votes · traceability …
├── verifiers/    ▸ verify_citations.py — pre-LLM citation check
├── references/   ▸ hunt-methodology · pattern-library · severity-baseline · sec-context/
└── templates/    ▸ output scaffolds — advisory · retrospective · pre-poc-gate …
```

# Key features

- Multi-model (haiku/sonnet/opus) for efficiency
- Orchestrator->(sub)agents for context management
- Deterministic scripts for mechanical work
- High-quality resources as references/inputs
- Live PoCs in Docker
- Two review modes
  - Full review for comprehensive audit of entire codebase
  - Change review (*date/commit/version)* for incremental reviews in time

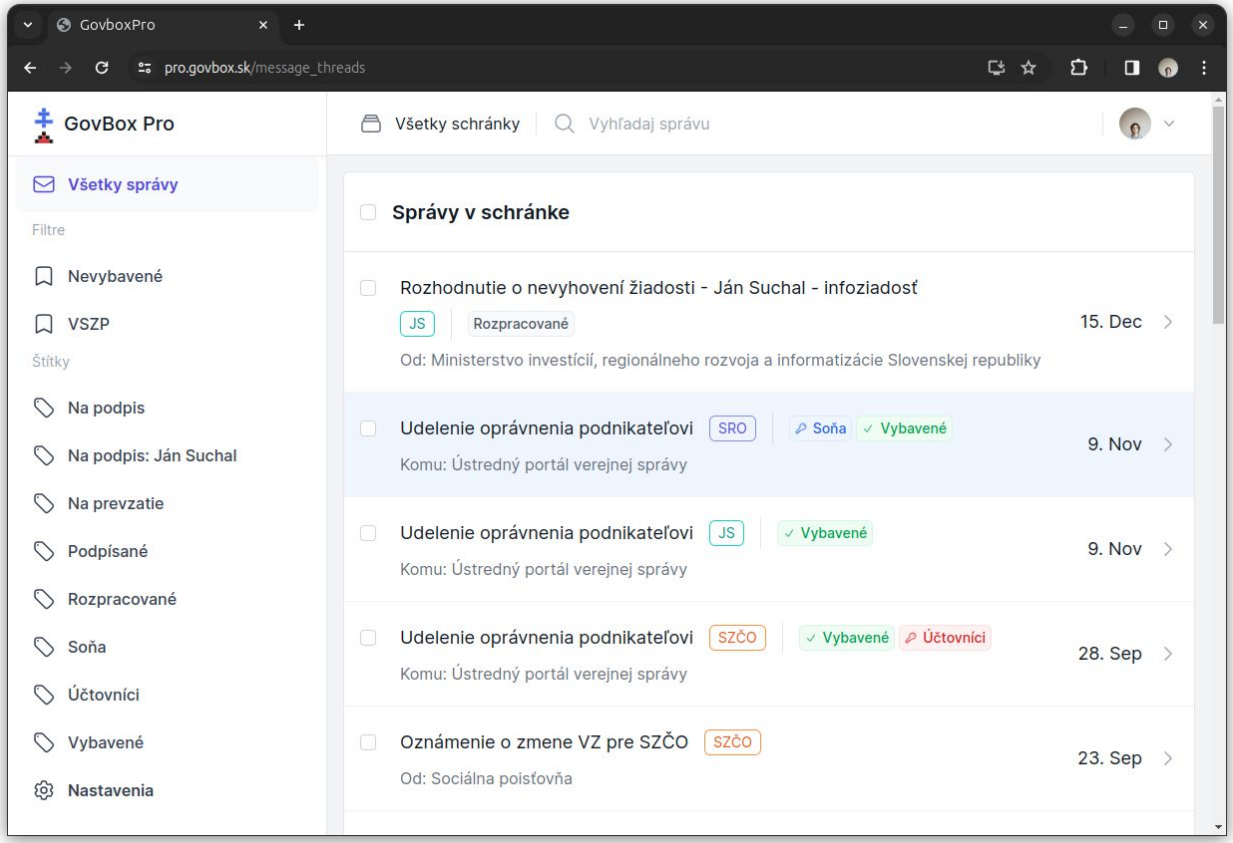
# Questions?

- Before we dive into the end-to-end example

# Case Study: GovBox Pro SaaS IDOR

# Context

- Webapp to manage messages with government
- Slovensko.Digital - Nonprofit org focused on better digital services in public sector
- 1x crit, 4x high, 2x medium
- kudos to Slovensko.Digital for the response!
- **“Tenant admin escalates to platform site-admin via cross-tenant duplicate email”**



# Attack scenario

1. Be tenant admin (register as customer)
2. Guess (bruteforce) site-admin email
3. Change your email to that email
4. Access GoodJob console
5. Manipulate jobs (sync, send, notify, …)
6. (to access message contents, use one of the IDORs/MAs)

# Root cause

**THE VULNERABLE CODE · three facts that combine**

app/models/user.rb:48 · config/routes.rb:344

```
def site_admin?
  ENV['SITE_ADMIN_EMAILS'].to_s.split(',').include?(email)   # authority keyed on a mutable email column
end
# /good_job (all-tenant job dashboard) mounted behind site_admin? only
```

db/schema.rb:723 · app/controllers/admin/users_controller.rb:55

```
# email is unique PER TENANT, not globally:
t.index ["tenant_id", "lower((email)::text)"], name: "index_users_on_tenant_id_and_lowercase_email",
unique: true

# a tenant admin can set ANY user's email (incl. their own):
params.require(:user).permit(:email, …)   # Admin::UserPolicy#update? = @actor.admin?  (no self-
exclusion)
```

# Recon

**PHASE 1 · RECON**

```
$ sed -n '30p' recon/recon-summary.md
```

- **site_admin?** (web) = email in `ENV['SITE_ADMIN_EMAILS']`; gates the GoodJob engine mount.

# Threat model

PHASE 2 · THREAT MODEL

```
$ sed -n '14p' recon/threat-model.md
```

- **good_job (DB-backed)** — 49 async jobs: external API ingestion, webhook firing, exports, document downloads. Dashboard mounted `/good_job` **(site-admin only)**.

```
$ sed -n '70p' recon/prior-vulns.md
```

- … Verified in recon: `/good_job` gated by `GoodJobAdmin.matches?` → `User.find(session[:user_id])&.site_admin?` (site-admin only). Phase 5: confirm gate is robust and **not tenant-crossing in job args**.

# Scan (Haiku) - IDOR + MA

**PHASE 3 · SCAN · per-file LLM (Haiku)**

```
$ jq -c '.candidates[0]' scan/haiku-results/app_policies_admin_api_connection_policy.rb.json
```

{"line":22,"vuln_class":"broken-object-authz","pattern":"destroy? returns @user.admin? **without verifying @api_connection.tenant**","confidence":"high","snippet":"@user.admin?"}

```
$ jq -c '.candidates[0]' scan/haiku-results/…_fs_api_connections_controller.rb.json
```

{"line":25,"vuln_class":"**mass-assignment**","pattern":"permit list includes :tenant_id in multi-tenant context; sensitive field should not be user-assignable","confidence":"medium","snippet":"permit(**:tenant_id**, :custom_name, :settings_username, :settings_password)"}

# Custom SAST rule - IDOR

**PHASE 4 · CUSTOM SEMGREP · synthesized rule (4b)**

```
$ sed -n '/patterns:/,/paths:/p' scan/custom-rules/idor-raw-find-then-authorize.yml
```

```
    patterns:
      - pattern-either:
          - pattern: "$MODEL.find(params[:id])"
          - pattern: "$MODEL.find_by(id: params[:id])"
      - metavariable-pattern:
          metavariable: $MODEL
          patterns:
            - pattern-not-regex: ^(policy_scope|Current|@tenant|@box|@group|accessible_boxes)
```

# Hunt

**PHASE 5 · HUNT**

```
$ jq '.[] | select(.id=="slice-4-5") | {id,file,line,severity,title}' scan/hunt-slice-4-findings.json
```


```
{
  "id": "slice-4-5",
  "file": "app/models/user.rb",
  "line": "44",
  "severity": "high",
  "title": "site_admin? authority derived from ENV['SITE_ADMIN_EMAILS'] matched against the same email an attacker can forge via the OAuth email-match login"
}
```

# Hunt - Variants

**PHASE 5 · HUNT · variant analysis (the self-contained path, no OAuth)**

amendments/F134-live.md › The new exploitation path

Three facts combine into a self-contained escalation:

1. **site_admin? ignores tenant** and is re-read from the DB each request.
2. **Email uniqueness is **tenant-scoped, not global**** — index (tenant_id, lower(email)). The same email can coexist in multiple tenants. (The `identities` table DOES have a global unique email index — **the protection exists, just on the wrong table**.)
3. **A tenant admin can **set any user's :email, including their own**** — Admin::UsersController#update permits :email; Admin::UserPolicy#update? is just @actor.admin?.

# Validate

**PHASE 6 · VALIDATE**

```
$ jq '.[] | select(.id=="F134") | {id,verdict,confidence,reasoning}' validation/batch-01-broken-authentication.json
```


```
{
  "id": "F134",
  "verdict": "confirmed",
  "confidence": "high",
  "reasoning": "… an attacker who can get any enabled IdP to assert a configured site-admin email (or who can match such an email cross-tenant) is authenticated as a User whose `email` equals a SITE_ADMIN_EMAILS value …"
}
```

# PoC

**PHASE 7 · POC · runtime (Docker, multi-tenant seed)**

```
$ bin/rails runner poc/f134_poc.rb           # → poc/poc-output.txt
```

```
[setup]  attacker tenant=4 user=4 admin?=true site_admin?(before)=false
[policy] Admin::UserPolicy#update?(self) = true
[attack] attacker.update(email: "admin@test.com") => saved=true errors=[]
[result] attacker.email=admin@test.com tenant=4 site_admin?(after)=true
[dup-check] users with email admin@test.com:  [1, 1, "admin@test.com"], [4, 4, "admin@test.com"]
>>> F134 SCENARIO-2 CONFIRMED
```

**PHASE 7 · POC · end-to-end over HTTP (full Rack stack)**

```
$ bin/rails runner poc/http_e2e.rb           # → poc/http-e2e-output.txt
```

```
[OAuth login callback] -> HTTP 302   session[user_id]=7 tenant_id=7
[GET /good_job BEFORE] -> HTTP 404            # GoodJobAdmin constraint false -> no route
[PATCH own user email -> admin@test.com] -> HTTP 302
[GET /good_job AFTER]  -> HTTP 301            # engine now mounted -> redirects into /good_job/jobs
>>> F134 HTTP CONFIRMED: /good_job 404->301 after self-email PATCH
```

# Report

PHASE 8 · REPORT

```
$ head -6 advisories/h1_report_privilege_escalation_critical.md
```

```
# Tenant admin escalates to platform site-admin via cross-tenant duplicate email

**Weakness**: CWE-639: Authorization Bypass Through User-Controlled Key (also CWE-266: Incorrect
Privilege Assignment)
**Severity**: critical — CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:H (9.9)
**Asset**: github.com/slovensko-digital/govbox-pro — deployed at pro.govbox.sk
```

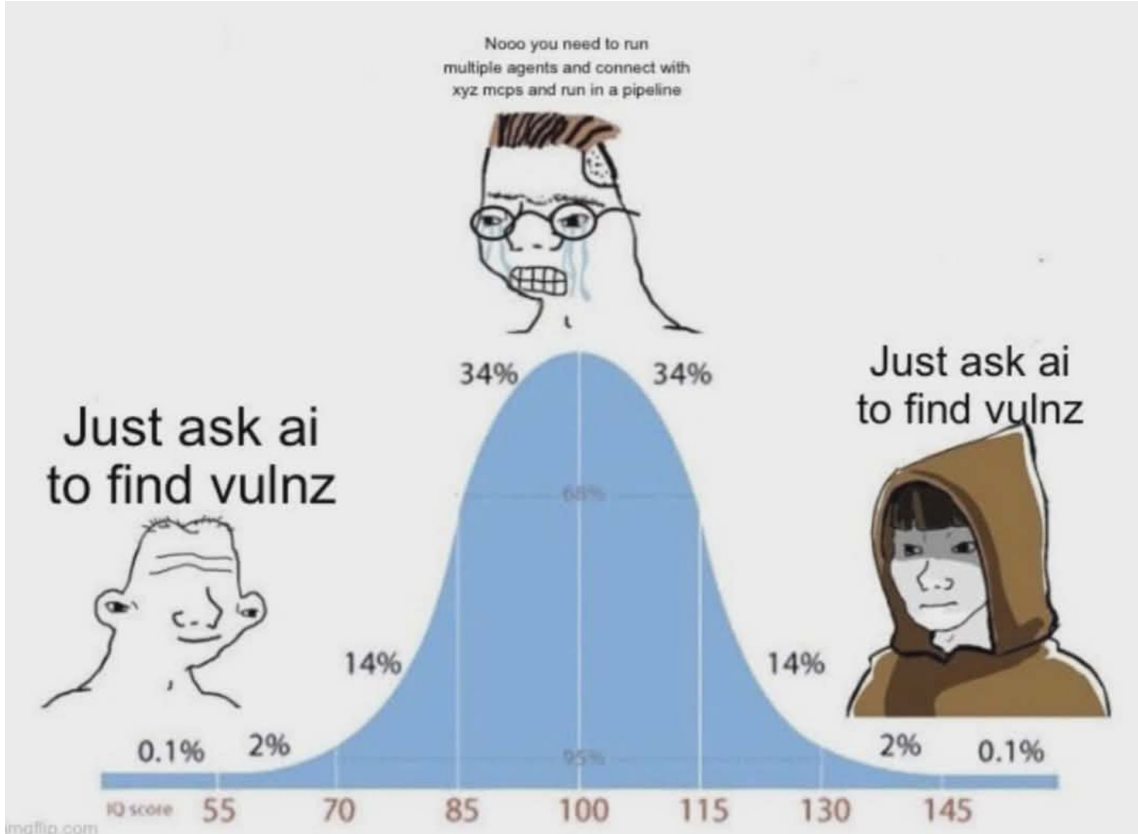
# Closing remarks

# Future work

- Improve the technical quality of the harness
  - Scripts, prompts, skills, commands (mostly) vibecoded
- Completeness/coverage
  - Current goal is to discover and report vuln, but pentesting requires more focus on coverage
- Add more vulnclass-specific instructions and novel approaches
- Add support for other providers, and eventually figure out local models
  - Anthropic won’t be subsidizing forever

# Lessons learned (on LLMs finding bugs)

- **Different is more important than better**
- LLMs are great for finding variants of existing stuff
  - Not that great in being creative for new approaches
  - Works nice for business logic bugs too!
- Intended feature vs vuln problem
- Work with real data (targets) ASAP
  - Public intentionally-vulnerable -> in the training data
  - Private intentionally-vulnerable projects -> different FP vs FN ratio
- The meme is true btw
  - “just ask” works great if you can ask better than others
  - https://matejsmycka.github.io/web/contents/vulns/



# Lessons learned (on harness design)

- No need for Langchain or other fancy tools to start!
- Great to combine LLMs with deterministic tools + completeness enforcement
- “meta-harness-development”
  - *“based on <guide from anthropic> and **previous session logs**, review the commands and skills and propose improvements”*
  - Better models -> better harnesses -> … -> …
- Context window is key
  - (sub)agents ftw!
- Repeated QA agents (fresh context + adversarial prompts)
  - Two iterations of “*/report_qa (make sure report line by line correct, poc executes copy&paste)”* do wonders to the report quality

# Lessons learned (on operations)

- Model providers are not your friends
  - The *“I won’t run the pipeline we ran for the past 3 months because it’s random Thursday morning and my creators are tweaking something”*
  - Remember what happened with Fable? Imagine being dependent on it
- Either direct cross-provider (+local) support, or ease to migrate when needed is crucial
- “Cheap” access to great quality models? Great, but it won’t last. Enjoy while we have it!
  - In one week, I ran ~25 reviews with a rough estimate of **~100$ per review in API costs**
  - Not too bad for 180€/month, right?
  - (need to doublecheck the numbers, looks way too cheap!)

# Future of LLM vuln research/pentest

- **IMHO**
- In the (truly) long run
  - better the model, more mistakes you can make
  - model providers will have this built-in in their harness, and projects will be checked automatically for 99% of vulns
  - Novel research done by humans using AI, adding the creativity AI/LLMs lack (for now?)
- In the meantime
  - great opportunity to complement traditional pentesting/appsec. AI might be overhyped, but it clearly provides new capabilities (take this project!)
  - there is still edge to be gained with a good harness (and cheap tokens)

# Acknowledgements & Thanks

- Similar OSS projects
  - remember *different* > *better*
- BB/pentesting resources (OWASP, Trail of Bits, J. Haddix, …)
- Anthropic, OpenAI, **AISLE,** … resources
- Any questions?